F1 StrategyIQ

Data-Driven Race Strategy Optimization & Dynamic Pit-Wall Decision Support System

Final Detailed Project Proposal
Data Analytics & Data Science Course Project

> **Core idea:** Use public F1 data to analyze race and tyre behavior, predict lap performance and degradation with ML, simulate alternative strategies, optimize a preferred strategy, and dynamically recommend actions such as PIT NOW or STAY OUT during a race replay.

1. Executive Summary

F1 StrategyIQ is an interactive data-science decision-support application inspired by the strategic questions handled on an F1 pit wall. Historical Formula 1 data is cleaned and stored in a structured database. Machine-learning models predict quantities such as lap time and tyre degradation. A race simulator evaluates candidate tyre/pit strategies under uncertainty, while an optimization layer selects a strong strategy based on expected performance, position, podium/win probability and risk. A dynamic Pit-Wall engine then re-evaluates the recommendation whenever the race state changes materially.

> **Positioning:** This is a public-data academic prototype, not a replica of proprietary F1 team software. Exact fuel load, tyre condition, setup, engine modes and private strategic information are unavailable publicly.

2. Problem Statement

F1 strategy is not one prediction problem. The system must estimate future pace, understand tyre degradation, account for pit loss and race position, compare several possible strategies, and adapt when the actual race differs from the original plan.

> The project therefore follows: **Predict** → **Simulate** → **Optimize** → **Decide**.

3. Complete Architecture

> **Historical F1 Sources** → **Data Ingestion** → **PostgreSQL** → **Cleaning & Feature Engineering** → **EDA/Statistics** → **ML Models** → **Race Simulator** → **Strategy Optimizer** → **Dynamic Pit-Wall Engine** → **FastAPI** → **Streamlit Dashboard**

During a race replay, the system forms a closed loop: **race state** → **prediction** → **simulation** → **decision** → **updated race state** → **re-evaluation**.

4. Data Sources & Scope

Source	Use
FastF1	Timing, laps, telemetry, positions, tyres, weather, track status and race-control data.
Jolpica F1 API	Structured historical race results, drivers, constructors, standings, laps and pit stops.
Public/Kaggle datasets	Optional baseline, validation and cross-checking.

Recommended experimental scope: 6–8 deliberately diverse circuits across multiple recent seasons. A practical starting range is 2021–2025, with season/regulation context included. Prefer a chronological/race-level split such as 2021–2024 training and 2025 testing rather than a random lap-level split.

Candidate circuits: Bahrain, Monaco, Silverstone, Spa, Monza, Singapore, Suzuka and Interlagos. The final set can be adjusted based on data completeness.

5. Database Design

Table	Purpose / main fields
races	race_id, season, round, race_name, circuit_id, date
circuits	circuit_id, name, country, city, location and circuit characteristics
drivers	driver_id, name, code, nationality
constructors	constructor_id, constructor_name
results	race_id, driver_id, constructor_id, grid, finish, points, status

Table	Purpose / main fields
laps	race_id, driver_id, lap_number, position, lap_time and sectors
tyre_stints	race_id, driver_id, stint, compound, start_lap, end_lap, tyre_age
pit_stops	race_id, driver_id, lap, pit_duration
weather	timestamp, air/track temperature, humidity, rainfall, wind
telemetry/features	Process large telemetry into derived features rather than storing every point in PostgreSQL.

Fuel limitation: exact fuel load is not public. Use defensible proxies such as lap/stint progression and historical pace trends; do not claim exact fuel state.

6. Feature Engineering

• Tyre age = current lap – tyre start lap + 1

• Rolling pace: previous 3/5 lap averages and pace trend

• Compound, stint number and remaining laps

• Position, grid position, gap ahead and gap behind

• Driver, constructor, circuit and season

• Track/air temperature and rainfall

• Safety Car/VSC and track-status indicators

• Pit duration / estimated pit loss

• Sector-time and selected telemetry-derived features

> **Data-leakage rule:** at a simulated decision point, only information available by that point may be used. Future laps, future pit stops and future race results cannot enter the features.

7. Machine-Learning Layer

A. **Lap-Time Prediction:** predict expected lap time from driver, constructor, circuit, tyre compound, tyre age, lap number, recent pace and weather/race-state features. Compare Linear Regression, Random Forest and XGBoost. Evaluate with MAE, RMSE and R^2.

B. **Tyre Degradation Prediction:** estimate expected pace loss as a stint ages. The model must acknowledge confounding effects such as traffic, fuel, driver behavior, track evolution, weather and flags.

C. **Pit Decision Classification:** classify the current state into actions such as PIT NOW or STAY OUT, with an optional PIT WINDOW/PIT NEXT category. Evaluate with precision, recall, F1 and ROC-AUC.

8. Race Simulation & Monte Carlo

The simulator evaluates feasible strategies such as Medium→Hard, Medium→Medium→Hard and Soft→Medium→Hard. It projects lap performance, tyre degradation and pit-loss effects across the remaining race.

Monte Carlo simulation can repeat the race many times while varying uncertain quantities such as degradation, traffic, Safety Car scenarios and model error. Outputs include expected race time, expected finish, podium probability, win probability and risk.

9. Strategy Optimization

The optimizer compares simulated strategies and chooses a preferred strategy using a documented objective. Fastest predicted race time alone is insufficient because track position, traffic, podium probability and risk can matter.

> **Validation:** compare recommendations against historical actual strategies and simple rule-based baselines. Do not call a strategy 'optimal' merely because the model selected it.

10. Dynamic Pit-Wall Module

This is the during-the-race component. It does not attempt to reproduce professional millisecond-by-millisecond live timing. Instead, it processes meaningful race-state updates, such as every lap or major events such as Safety Car, competitor pit stops or substantial condition changes.

> **Example state:** Lap 28 | P5 | Medium | Tyre age 18 | Gap ahead 2.1s | Gap behind 1.4s | High degradation | Safety Car ON
>
> **Possible decision:** PIT NOW — confidence 84%. Reasons may include high degradation, reduced pit-loss under Safety Car, sufficient gap protection and projected fresh-tyre advantage.

For demonstration, an old race can be replayed lap-by-lap. Each historical state is fed into the decision engine, so the dashboard visibly shows recommendations changing as the race evolves.

11. Interactive Frontend

The frontend will be a **Streamlit interactive dashboard**, not a text-only output screen.

Page	Interactive/visual content
Race Overview	GP, driver, lap, position, tyre, gaps and key KPIs.
Race Analysis	Position changes, lap-time trends, sector performance and comparisons.
Tyre Intelligence	Tyre-age curves, degradation trends, compound comparisons and projected window.
Strategy Optimizer	Candidate strategies, expected time, finish distribution, podium/win probability and risk.
Dynamic Pit-Wall	Current state, PIT NOW/STAY OUT, confidence and reasons.
What-If Simulator	Change pit timing, tyre choice, Safety Car/weather assumptions and risk preference.

Key visuals: race-position chart, actual-vs-predicted lap-time graph, tyre degradation curves, pit-window timeline, strategy comparison bars, Monte Carlo finishing-position distribution and KPI cards.

12. Backend & API

FastAPI connects the Streamlit frontend to the database, ML models and strategy engine. Models are trained offline and saved with Joblib/Pickle; the API loads them instead of retraining for every request.

Endpoint	Purpose
GET /api/races	List available races
GET /api/races/{race_id}	Return race-level data/state
GET /api/drivers/{driver_id}/performance	Driver performance analysis
POST /api/predict/lap-time	Predict lap performance
POST /api/predict/tyre	Predict tyre degradation/pace loss
POST /api/strategy/optimize	Run simulation and strategy optimization
POST /api/pitwall/decision	Return PIT NOW/STAY OUT recommendation

13. Input → Processing → Output

Input	Processing	Output
Race/driver/starting tyre	Historical retrieval + models	Race analysis + tyre intelligence
Lap/tyre age/pace/weather/gaps	Prediction models	Predicted lap time + degradation
Candidate strategies	Simulation + Monte Carlo	Expected time, finish, podium/win probability, risk
Current race state	Dynamic decision engine	PIT NOW / STAY OUT + confidence + explanation
What-if controls	Re-run scenario	Visual alternative-strategy comparison

14. Technology Stack

Layer	Technology
Data	FastF1, Jolpica F1 API, Pandas, NumPy

Layer	Technology
Statistics	SciPy / Statsmodels
Visualization	Plotly, Matplotlib, Seaborn
ML	Scikit-learn, XGBoost
Database	PostgreSQL
Backend	FastAPI
Frontend	Streamlit
Models	Joblib / Pickle
Version control	Git + GitHub

15. Three-Month Implementation Plan

Weeks	Work
1–2	Finalize scope; FastF1/Jolpica ingestion; select circuits/seasons; PostgreSQL schema.
3–4	Cleaning, feature tables, EDA, statistics and visualizations.
5–6	Lap-time regression; compare baseline, Random Forest and XGBoost.
7–8	Tyre degradation model and pit-decision classifier; evaluation.
9–10	Race simulator, candidate strategy generator and Monte Carlo.
11	Dynamic Pit-Wall engine and FastAPI.
12	Streamlit integration, testing, report, PPT and viva preparation.

16. Evaluation

- **Regression:** MAE, RMSE, R^2.
- **Classification:** precision, recall, F1, ROC-AUC.
- **Strategy:** expected race time, finishing-position distribution, podium/win probability and risk.
- **Validation:** historical replay plus rule-based baselines.
- **Generalization:** chronological/race-level split to reduce leakage.

17. Major Limitations & Mitigation

Limitation	Mitigation
Exact fuel load unavailable	Use transparent proxies.
No proprietary tyre/setup/engine data	Clearly position as public-data prototype.
Gaps change rapidly	Use lap/event-based updates, not millisecond streaming.
Weather changes unexpectedly	Recalculate using observed/scenario conditions and show uncertainty.
Safety Car changes strategy	Use Safety Car/VSC as race-state and simulation variables.
Actual teams may differ	Expected because private information is unavailable.
Model can be wrong	Show confidence/uncertainty and benchmark against baselines.

18. MVP vs Stretch Goals

MVP: 6–8 circuits × multiple recent seasons → clean database → EDA → lap/tyre prediction → strategy simulator → strategy comparison → Streamlit dashboard.

Stretch: dynamic Pit-Wall replay, Monte Carlo uncertainty, what-if simulation, weather/Safety-Car scenario analysis and richer telemetry features.

Avoid unless time remains: full reinforcement learning, computer vision, true live streaming, full physics simulation or deep learning everywhere.

19. Final Demonstration

1. Select a historical GP and driver.
2. View race/tyre analysis.
3. Run the strategy optimizer.
4. Compare candidate strategies visually.
5. Replay the race lap-by-lap in Dynamic Pit-Wall.
6. Demonstrate PIT NOW versus STAY OUT.
7. Change assumptions in the What-If simulator.
8. Explain metrics, uncertainty, limitations and validation.

20. Final One-Minute Explanation

> “F1 StrategyIQ is an interactive data-driven Formula 1 strategy decision-support system. We collect historical F1 data and store it in a structured database. Machine-learning models predict lap performance and tyre degradation, then a race simulator and Monte Carlo analysis evaluate different pit-stop and tyre strategies. An optimization layer selects a strong strategy based on expected performance and risk. During a race replay, the dynamic Pit-Wall

engine receives the current race state and recommends actions such as PIT NOW or STAY OUT when meaningful conditions change. The results are presented through an interactive Streamlit dashboard with race charts, tyre-degradation graphs, strategy comparisons, probability distributions and what-if simulations, with FastAPI connecting the frontend to the ML and strategy backend.”

21. Core Principle

Analyze → **Predict** → **Simulate** → **Optimize** → **Decide** → **Re-evaluate**